Pointers and Arrays

30 October, 2022

Question

- What is the difference between the *arr* we get from
  - int arr[10]
  - int *arr followed by arr = malloc(sizeof(int)*10)
- Both are pointers to the first block of memory assigned to the array
- Static assigned pointer cannot be re-assigned
- Dynamically assigned pointer can be re-assigned

Arrays and Pointers

- In C, array names are nothing but pointers.
  - Can be used interchangeably in most cases
- However, array names can not be assigned, but pointer variables can be.

```
int ar[10], *b;
ar = ar + 2;
ar = b;
b = ar;
b = b + 1;
b = ar + 2;
b++;
```








Array of Pointers

- Consider the following declaration

  int *arr[10];

- arr is a 10-sized array of pointers to integers
- How can we have equivalent dynamic array?

```
int **arr;
arr = (int**) malloc ( 10 * sizeof(int *) );
```

Array of Pointers

```
int **arr;
arr = (int**) malloc ( 10)* sizeof(int *)
```

- Note that individual elements in the array arr (arr[0], … arr[9]) are NOT allocated any space. Uninitialized.
- We need to do it (directly or indirectly) before using them.

```
int j;
for (j = 0; j < 10; j++)
    arr[j] = (int*) malloc (sizeof(int));
```

Exercise: All Substrings

- Read a string and create an array containing all its substrings (i.e. contiguous).
- Display the substrings.

Input: ESC

Output:

E

ES

ESC

S

SC

C

All Substrings: Solution Strategy

- What are the possible substrings for a string having length n?
- Allocate a 2D char array having K= n(n+1)/2 rows (Why ? How many columns?)
- Copy the substrings into different rows of this array.

```
int len, i, j, k=0, nsubstr;
char st[100], **substrs;
scanf("%s",st);
len = strlen(st);
nsubstr = len*(len+1)/2;
substrs = (char**)malloc(sizeof(char*) * nsubstr);
for (i=0; i<nsubstr; i++)
    substrs[i] = (char*)malloc(sizeof(char) * (len+1));

for (i=0; i<len; i++){
    for (j=i; j<len; j++){
        strncpy(substrs[k], st+i, j-i+1);
        k++;
    }
}
for (i=0; i<k; i++)
    printf("%s\n",substrs[i]);
```

```
for (i=0; i<k; i++)
    free(substrs[i]);
free(substrs);
```

Too much wastage…

E	**‘\0’**		
E	**S**	**‘\0’**	
E	**S**	**C**	**‘\0’**
S	**‘\0’**		
S	**C**	**‘\0’**	
C	**‘\0’**		

Solution: Version 2

```
int len, i, j, k=0,nsubstr; char st[100], **substrs;
scanf("%s",st);
len = strlen(st);
nsubstr = len*(len+1)/2;
substrs = (char**)malloc(sizeof(char*) * nsubstr);

for (i=0; i<len; i++)
   for (j=i; j<len; j++){
      substrs[k] = (char*)malloc(sizeof(char) * (j-i+2));
      strncpy(substrs[k], st+i, j-i+1);
      k++;
    }
for (i=0; i<k; i++)
    printf("%s\n",substrs[i]);
```

```
for (i=0; i<k; i++)
    free(substrs[i]);
free(substrs);
```

This version uses much less memory compared to version 1

Example Function that Returns Pointer

char *strdup(const char *s);

- strdup creates a copy of the string (char array) passed as arguments
  - copy is created in dynamically allocated memory block of sufficient size
- returns a pointer to the copy created

char *strndup(const char *s, size_t num);

- Returns a pointer to a null-terminated byte string, which contains copies of at most num bytes from the string pointed to by s.

Solution: Version 3

```
int len, i, j, k=0,nsubstr;
char st[100], **substrs;
scanf("%s",st);
len = strlen(st);
nsubstr = len*(len+1)/2;
substrs = (char**)malloc(sizeof(char*) * nsubstr);

for (i=0; i<len; i++){
   for (j=i; j<len; j++){
      substrs[k] = strndup(st+i, j-i+1));
      k++;
    }
}
for (i=0; i<k; i++)
    printf("%s\n",substrs[i]);
```

```
for (i=0; i<k; i++)
    free(substrs[i]);
free(substrs);
```

Less code => more readable, fewer bugs!
possibly faster!